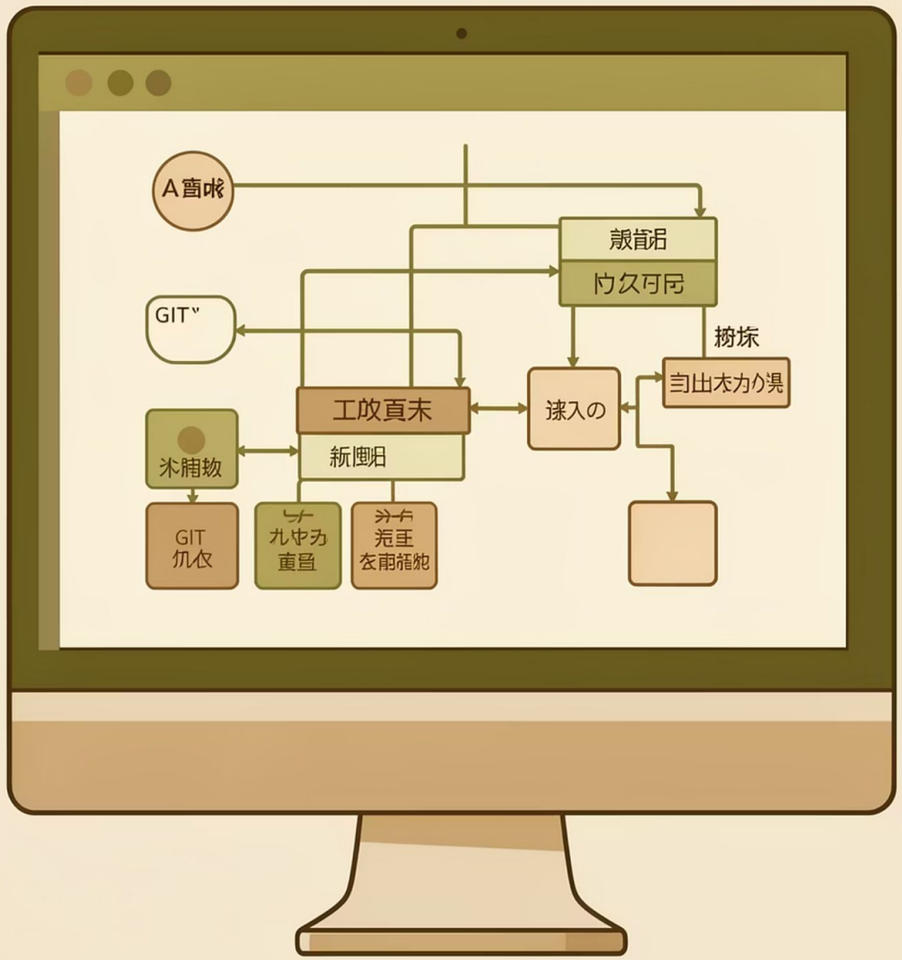
Introduzione a Git Remoto e Repository Online

Impara a gestire il tuo codice collaborativamente e in modo professionale



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Git Locale vs Git Remoto: La Differenza Fondamentale

Repository Locale

Il tuo repository locale vive esclusivamente sulla tua macchina. Ogni commit, branch e modifica rimane confinato al tuo computer. È perfetto per lavorare offline e sperimentare senza influenzare gli altri.

- Accesso solo da una macchina
- Nessuna condivisione automatica
- Rischio di perdita dati se il PC si rompe

Repository Remoto

Un repository remoto risiede su un server esterno, accessibile da chiunque abbia le credenziali. È il centro di coordinamento dove tutti i collaboratori sincronizzano il loro lavoro.

- Accesso da qualsiasi dispositivo
- Condivisione e collaborazione in tempo reale
- Backup automatico e sicurezza

© 2026 Mattia Nardulli – Tutti i diritti riservati.

Perché Usare un Repository Remoto?



Backup Automatico

Il tuo codice è al sicuro su server affidabili. Anche se il tuo computer si rompe, la tua storia di commit e tutti i tuoi progetti sono preservati nel cloud.



Collaborazione in Tempo Reale

Lavora simultaneamente con colleghi e amici. Merge, pull requests e code review diventano semplici. Ogni membro del team può contribuire senza conflitti.



Integrazione CI/CD

Automatizza test, build e deployment ogni volta che pushi nuove modifiche. Migliora la qualità del codice e riduci gli errori prima della produzione.



Storia Completa

Mantieni traccia di ogni modifica, chi l'ha fatta e quando. Ripristina versioni precedenti in pochi secondi e analizza l'evoluzione del progetto nel tempo.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

HTTPS vs SSH: Scegliere il Protocollo Giusto

HTTPS

Il protocollo più semplice e universale. Funziona ovunque senza configurazione aggiuntiva e utilizza le tue credenziali di GitHub/GitLab per l'autenticazione.

Vantaggi

- Facile da configurare per i principianti
- Funziona dietro qualsiasi firewall
- Non richiede chiavi SSH

Contro

- Devi inserire username e password ogni volta
- Menù sicuro di SSH

SSH

Il protocollo più sicuro e professionale. Usa coppie di chiavi crittografiche per l'autenticazione automatica senza password.

Vantaggi

- Autenticazione automatica con chiavi
- Maggiore sicurezza crittografica
- Nessun inserimento credenziali

Contro

- Configurazione iniziale più complessa
- Può essere bloccato da alcuni firewall



Suggerimento: Inizia con HTTPS per semplificare, poi passa a SSH quando ti senti più sicuro!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Configurazione accesso tramite HTTPS

```
git config --global user.name "IlTuoNome"
git config --global user.email "latuaemail@example.com"
```



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

Il Flusso di Lavoro Git: Un'Immagine Vale Mille Parole

Commit Locale

Push al Server

Distribuzione

1. Il Tuo Computer

Clona il repository remoto sul tuo locale. Lavora, committa le modifiche e testa in totale sicurezza prima di condividere.

2. Il Server Remoto

Pusha le tue modifiche al repository centrale. Qui il codice viene salvato, versionato e reso disponibile a tutti.

3. I Collaboratori

Gli altri team member pullano le tue modifiche e contribuiscono con le loro. La sincronizzazione avviene continuamente.



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

Git in Azione: GitHub e GitLab

GitHub

La piattaforma più popolare per progetti open source. Oltre 100 milioni di utenti collaborano su milioni di repository. Offre Issues, Pull Requests, Actions per CI/CD, e una community enorme.

- Interfaccia intuitiva e moderna
- Actions per automazione
- Pages per hosting statico
- Community e documentazione

GitLab

Una soluzione completa che include CI/CD integrato, issue tracking e DevOps tools. Perfetto per team che vogliono tutto in un'unica piattaforma.

- CI/CD integrato nativamente
- Funzionalità DevOps complete
- Versione self-hosted disponibile
- Strumenti di sicurezza avanzati

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Branch Remoti: Collaborare Senza Conflitti

Crea Branch Remoto

Quando lavori su una nuova funzionalità, crea un branch dedicato. Pushalo sul remoto per condividerlo con il team.

Collabora Separatamente

I tuoi colleghi possono lavorare sui loro branch mentre tu sviluppi la tua feature. Nessun conflitto con il codice principale.

Merge Quando Sei Pronto

Una volta completata la feature, apri una Pull Request per revisionare il codice e mergiare nel main branch in modo controllato.

Comandi Utili

```
git checkout -b feature/new-ui
git push origin feature/new-ui
git branch -r
```

Best Practices

- Nome chiari e descrittivi per i branch
- Pull regolarmente da main
- Delete branch dopo il merge

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Configurazione Iniziale: I Passi Fondamentali

Configura Git Globalmente

Imposta il tuo nome e email una volta per tutte. Queste informazioni verranno associate a ogni commit che farai.

```
git config --global user.name "Il Tuo Nome"
git config --global user.email "email@example.com"
```

Salvataggio credenziali HTTPS (Opzionale)

Automatizza l’inserimento delle credenziali.

```
git config --global credential.helper manager-core
```

Clona il Repository

Scarica il repository remoto sul tuo computer. Usa l'URL HTTPS o SSH fornito dalla piattaforma (GitHub, GitLab, Bitbucket).

```
git clone https://github.com/utente/progetto.git
cd progetto
```

Fai il Tuo Primo Commit

Modifica i file, aggiungili con add e crea il primo commit. Poi pusha sul remoto per condividere il tuo lavoro con il mondo!

```
git add .
git commit -m "Primo commit"
git push origin main
```

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Push, Pull e Fetch in Git

Comprendere i comandi fondamentali per la sincronizzazione tra repository locale e remoto

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl – Via Schiaparelli 14 - Torino https://www.tc-web.it

Terminologia Chiave di Git

1

Push

Invia le tue modifiche locali al repository remoto. È come dire: "Ho fatto del lavoro, condividiamolo con il team!"

```
git push origin main
```

Questo comando invia i tuoi commit al branch main del repository remoto chiamato origin.

2

Pull

Scarica le modifiche dal repository remoto e le integra nel tuo locale. Rimani sempre aggiornato con il lavoro degli altri.

```
git pull origin main
```

Combina fetch e merge in un solo comando per semplificare la sincronizzazione.

3

Fetch

Scarica le informazioni sui cambiamenti remoti senza modificarlo ancora il tuo codice locale. Permette di vedere cosa è cambiato.

```
git fetch origin
```

Dopo il fetch, puoi decidere quando e come integrare le modifiche con merge o rebase.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting



git push: Caricare le Tue Modifiche

Il comando **git push** è il tuo strumento per condividere il lavoro completato con il team. Quando hai finito di lavorare su una funzionalità o hai corretto un bug, usa push per inviare i tuoi commit al repository remoto.

Sintassi di Base

```
git push <remote> <branch>
```

Esempi comuni:

```
git push origin main
git push origin develop
git push origin feature/new-login
```

Quando Usare Push

- Completamento di una feature o bug fix
- Al termine di una sessione di lavoro
- Dopo aver risolto i conflitti
- Per backup automatico del tuo lavoro

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Flusso di git push



Quando esegui **git push**, Git confronta la cronologia dei commit locali con quella remota. Se la tua cronologia è più recente, Git carica i nuovi commit sul repository remoto. Se invece il remoto ha commit più recenti, Git rifiuterà il push per evitare perdita di lavoro.

Attenzione: Se qualcun altro ha pushato modifiche dopo che hai clonato il repository, il tuo push verrà rifiutato. In questo caso, devi prima eseguire **git pull** per integrare le modifiche remote.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

git pull: Aggiornare il Lavoro Locale

Il comando **git pull** è una scorciatoia conveniente che combina due operazioni: **git fetch** (scarica le modifiche) e **git merge** (applica le modifiche). È il modo più comune per mantenere il tuo repository locale sincronizzato.

01

Scaricamento

Git scarica tutti i nuovi commit dal repository remoto

02

Integrazione

Git automaticamente unisce (merge) le modifiche remote con il tuo branch locale

03

Risultato

Il tuo repository locale è ora aggiornato con tutte le modifiche remote

Esempio di Terminale

```
git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0)
Unpacking objects: 100% (3/3), done.
From github.com:user/project
 * branch main -> FETCH_HEAD
 1a2b3c4..5d6e7f8 main -> origin/main
Updating 1a2b3c4..5d6e7f8
Fast-forward
 README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

git fetch: Esplorare le Modifiche Remote

Come Funziona

Il comando **git fetch** scarica le informazioni sulle modifiche remote senza modificarne il tuo repository locale. I commit vengono memorizzati in una branch remota (es. origin/main) ma non vengono automaticamente applicati alla tua branch corrente.

Sintassi

```
git fetch origin

git fetch origin main

git fetch --all
```

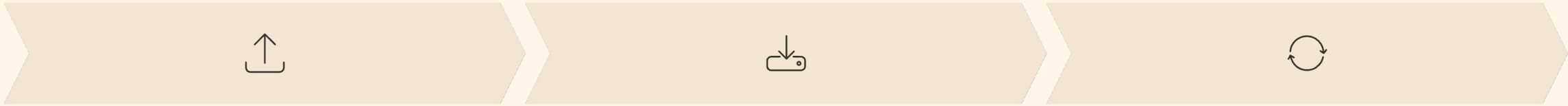
Quando Usare Fetch

- Vuoi vedere cosa hanno fatto gli altri senza modificare il tuo lavoro
- Devi esaminare le modifiche prima di applicarle
- Vuoi evitare merge automatici che potrebbero causare conflitti
- Stai lavorando in un ambiente dove la cautela è essenziale

Consiglio Pratico: Usa **git fetch** seguito da **git diff origin/main** per vedere le differenze prima di decidere se fare merge. Poi esegui **git merge origin/main** per applicare le modifiche.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Push vs Pull vs Fetch



git push

Direzione: Locale → Remoto

Effetto: Carica i tuoi commit

Rischi: Può essere rifiutato se il remoto è avanti

git pull

Direzione: Remoto → Locale

Effetto: Fetch + Merge automatico

Rischi: Può creare conflitti durante il merge

git fetch

Direzione: Remoto → Locale (copia remota)

Effetto: Solo download, nessuna modifica

Rischi: Nessuno, completamente sicuro

La scelta tra questi comandi dipende dal tuo workflow e dal livello di controllo che desideri. **git pull** è conveniente per la maggior parte dei casi, mentre **git fetch** offre maggiore controllo e consapevolezza.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Esercizio Guidato: Workflow Completo

Seguiamo un esempio pratico che simula una situazione reale di collaborazione. Questo esercizio ti aiuterà a comprendere come i tre comandi lavorano insieme nel flusso di lavoro quotidiano.

Creare Repository Locale

```
mkdir my-project
cd my-project
git init
echo "Hello World" > README.md
git add README.md
git commit -m "Initial commit"
```

Collegare Repository Remoto

```
git remote add origin https://github.com/user/my-project.git
git remote -v
git pull origin main --allow-unrelated-histories
```

Primo Push

```
git branch -M main
git push -u origin main
# -u imposta il tracking upstream
```

Simulare Modifiche Remote

(Apri il repository su GitHub web e modifica README.md aggiungendo una riga)

Eseguire Pull Locale

```
git pull origin main
# Git scarica e merge le modifiche
# Potrebbe creare un commit di merge
```

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Best Practices e Consigli Pratici

Fai Sempre Pull All'Inizio

Prima di iniziare una nuova sessione di lavoro, esegui **git pull** per assicurarti di partire dalla versione più recente del codice. Questo riduce drasticamente la probabilità di conflitti.

Usa Fetch per Esaminare le Modifiche

Se sei in dubbio su cosa è cambiato, usa **git fetch** seguito da **git log origin/main** o **git diff origin/main** per vedere le modifiche prima di applicarle.

Commit Piccoli e Frequenti

Commits più piccoli rendono i conflitti più facili da risolvere. Invece di un grande commit, fai più commit mirati. Questo semplifica anche il debugging.

Comunica con il Team

Quando lavori su file che altri stanno modificando, comunicalo nel tuo team. Strumenti come pull request e code review aiutano a coordinare le modifiche.

Risolvi i Conflitti Attivamente

Quando Git segnala un conflitto, apri il file, cerca le sezioni contrassegnate con `<<<<<<<`, `=======` e `>>>>>>>`, risolvi manualmente e poi esegui **git add** e **git commit**.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

GitHub Essentials: .gitignore, Pull Request e Issues

Ora è il momento di fare il salto verso i **workflow professionali su GitHub**. Impareremo a proteggere il repository con `.gitignore`, a collaborare in team tramite **Pull Request** e a gestire il lavoro con le **Issues**.



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

Cos'è il File .gitignore e Perché è Fondamentale

Il problema

Quando lavori su un progetto, la tua cartella è piena di file che **non devono mai finire su GitHub**: credenziali segrete, dipendenze installate localmente, file generati automaticamente. Senza controllo, questi file vengono inclusi nei commit per errore.

Chiavi API e password nel file `.env`

Cartella `node_modules` con migliaia di file

File di build come `/dist` o `/out`

File di configurazione IDE (`.idea/`, `.vscode/`)

La soluzione

Il file `.gitignore` è un file di testo che dice a Git **quali file e cartelle ignorare completamente**. Va creato nella root del progetto e committato nel repository, così tutti i collaboratori applicano le stesse regole.

Regola d'oro: Se un file può essere generato automaticamente o contiene segreti, non deve mai essere tracciato da Git.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

Esempio Pratico di .gitignore per un Progetto Web

Ecco un esempio commentato e realistico per un progetto Node.js/React. Ogni sezione copre una categoria diversa di file da escludere.

```
# ========================
# Dipendenze
# ========================
node_modules/
.pnp
.pnp.js

# ========================
# File di ambiente (SEGRETI)
# ========================
.env
.env.local
.env.production

# ========================
# Build e output
# ========================
/build
/dist
/.next
/out

# ========================
# File di sistema e IDE
# ========================
.DS_Store
Thumbs.db
.idea/
.vscode/
*.log
```

File già tracciati

Se aggiungi un file al `.gitignore` ma Git lo sta **già tracciando**, il file continuerà ad apparire nei commit. Devi rimuoverlo dalla cache con:

```
git rm --cached nomefile
```

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



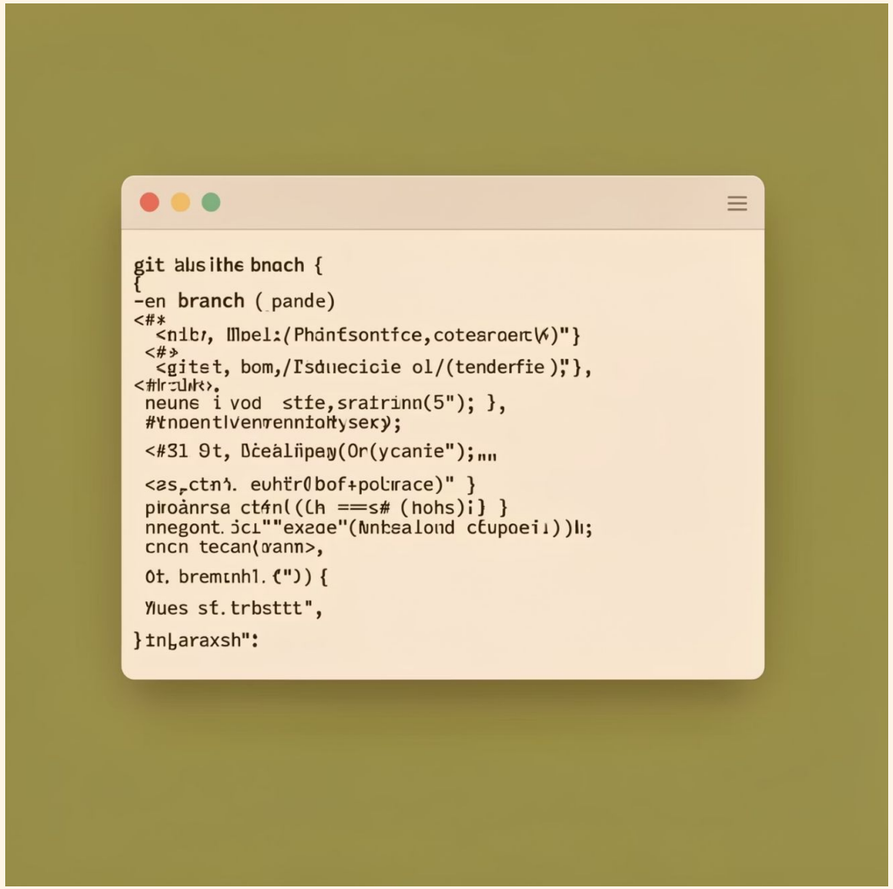
Collaborazione su Branch e Gestione dei Conflitti in Git Remoto

Lavorare in team con Git: dalla creazione di branch alla risoluzione di conflitti e best practice collaborative.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Creare e Pubblicare Branch su Remoto

I branch sono essenziali per lo sviluppo parallelo: permettono a più persone di lavorare su funzionalità diverse senza interferire tra loro. Creare un branch locale è solo il primo passo — devi anche pubblicarlo sul repository remoto per condividerlo con il team.



Comandi Fondamentali

`git branch nome-branch` — Crea un nuovo branch

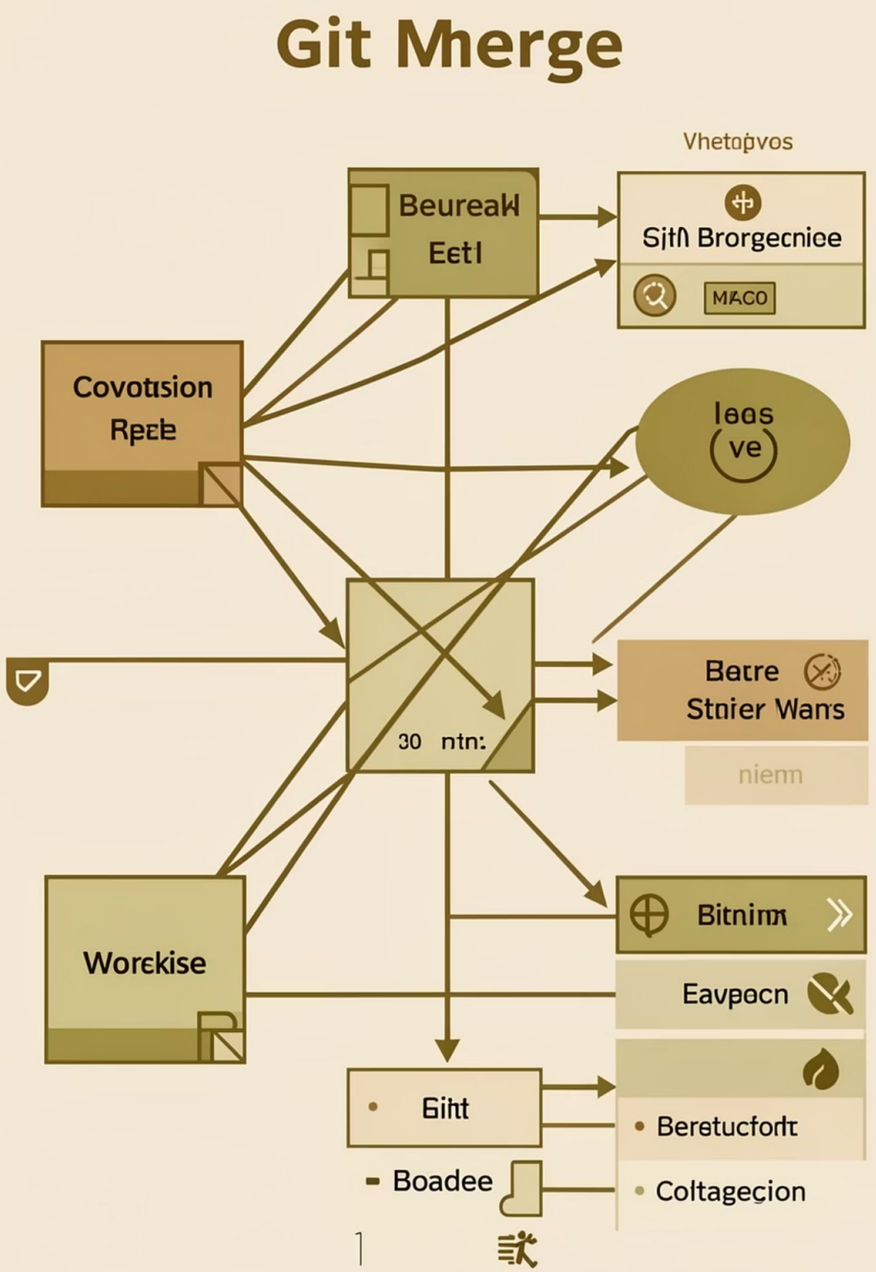
`git switch -c nome-branch` — Crea e passa al branch (forma compatta)

`git push origin nome-branch` — Pubblica il branch remoto

`git push -u origin nome-branch` — Imposta upstream automatico

Attenzione: Il flag `-u` (o `--set-upstream`) collega il branch locale a quello remoto, permettendo di usare semplicemente `git push` e `git pull` senza specificare ogni volta il nome del branch.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Aggiornare un Branch con le Modifiche di Main

Durante lo sviluppo, il branch `main` può ricevere nuove modifiche da altri collaboratori. Per mantenere il tuo branch aggiornato e ridurre i conflitti futuri, devi periodicamente integrare le ultime modifiche di `main` nel tuo branch di lavoro.

1

Passo 1: Cambia Branch

`git switch nome-branch`

Assicurati di essere nel branch corretto

2

Passo 2: Fetch e Merge

`git pull origin main`

Scarica e integra le modifiche

3

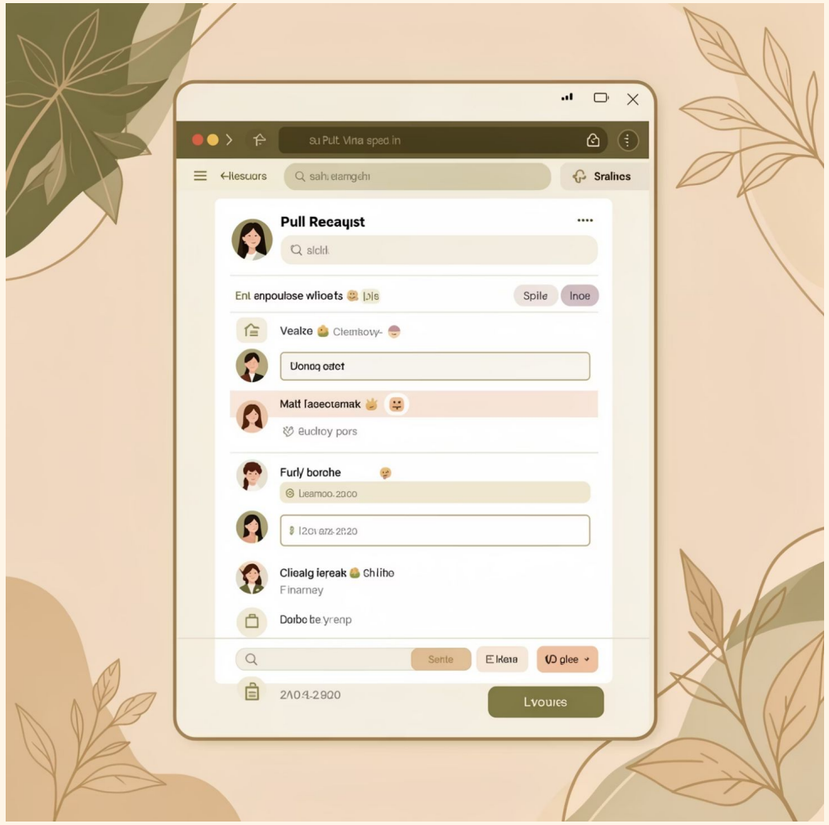
Passo 3: Risolvi Conflitti

Se necessario, risolvi eventuali conflitti prima di procedere

Questo processo mantiene il tuo branch sincronizzato con lo stato più recente del progetto, facilitando il merge finale.

© 2026 Mattia Nardulli - Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it
TC-Web Technology Consulting

Pull Request e Merge Request: Collaborazione Controllata



Cos'è una Pull Request?

Una Pull Request (PR) o Merge Request (MR) è una proposta formale di integrare le modifiche di un branch in un altro (solitamente `main`). Non è un comando Git, ma una funzionalità delle piattaforme di hosting come GitHub, GitLab o Bitbucket.

Flusso Tipico

1. Sviluppa la feature in un branch dedicato
2. Pubblica il branch remoto
3. Crea una PR sulla piattaforma
4. Altri sviluppatori revisionano il codice
5. Si discutono eventuali modifiche
6. Si approva e si merge la PR

Benefici: Le PR permettono revisione del codice (code review), discussioni su modifiche, test automatici prima del merge, e una tracciabilità completa di ogni cambiamento.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Best Practice per la Collaborazione Efficace

Nomi Chiari e Descrittivi

Usa convenzioni come `feature/descriptive-name`, `bugfix/issue-description`, o `docs/update-guide`. Evita nomi generici come `new-feature` o `fix`.

Commit Piccoli e Focalizzati

Fai commit frequenti con modifiche limitate e coerenti. Ogni commit dovrebbe rappresentare una modifica logica completa. Scrivi messaggi chiari e descrittivi.

Aggiorna Sempre Prima di Push

Esegui `git pull origin main` (o `git merge main`) nel tuo branch prima di pushare modifiche. Riduce drasticamente i conflitti e mantiene il branch sincronizzato.

Testa Prima di Push

Assicurati che il codice compili e passi i test prima di pubblicare le modifiche. Non pushare codice rotto su branch condivisi.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Cosa Sono le Issues e Perché Si Usano

Le **Issues** sono lo strumento di GitHub per tracciare tutto ciò che deve essere fatto in un progetto: bug da correggere, nuove funzionalità da aggiungere, domande, migliorie. Sono il punto di partenza di ogni attività di sviluppo professionale.



Bug Report

Qualcosa non funziona come dovrebbe. La Issue descrive il comportamento attuale, quello atteso e i passi per riprodurre il problema.



Feature Request

Una nuova funzionalità da sviluppare. La Issue descrive l'obiettivo, il valore per l'utente e i criteri di accettazione.



Label e Milestone

Le etichette categorizzano le Issues (`bug`, `enhancement`, `documentation`). Le milestone raggruppano Issues in sprint o versioni del prodotto.



Assegnazione

Ogni Issue può essere assegnata a uno o più sviluppatori. Chi è assegnato è responsabile di quella attività.

© 2026 Mattia Nardulli – Tutti i diritti riservati.

Collegare Issues e Pull Request: la Magia di "Closes #7"

GitHub permette di **collegare automaticamente** una Issue a una Pull Request usando parole chiave speciali nel messaggio di commit o nella descrizione della PR. Quando la PR viene mergiata, la Issue si chiude da sola.

Parole chiave supportate

Inseriscile nel messaggio di commit o nella descrizione della PR:

```
fixes #3
closes #7
resolves #12
```

Funzionano anche in inglese e italiano. GitHub riconosce automaticamente il numero della Issue e crea il collegamento.

Esempi concreti

```
# In un messaggio di commit:
git commit -m "Aggiunge validazione form
login - fixes #14"

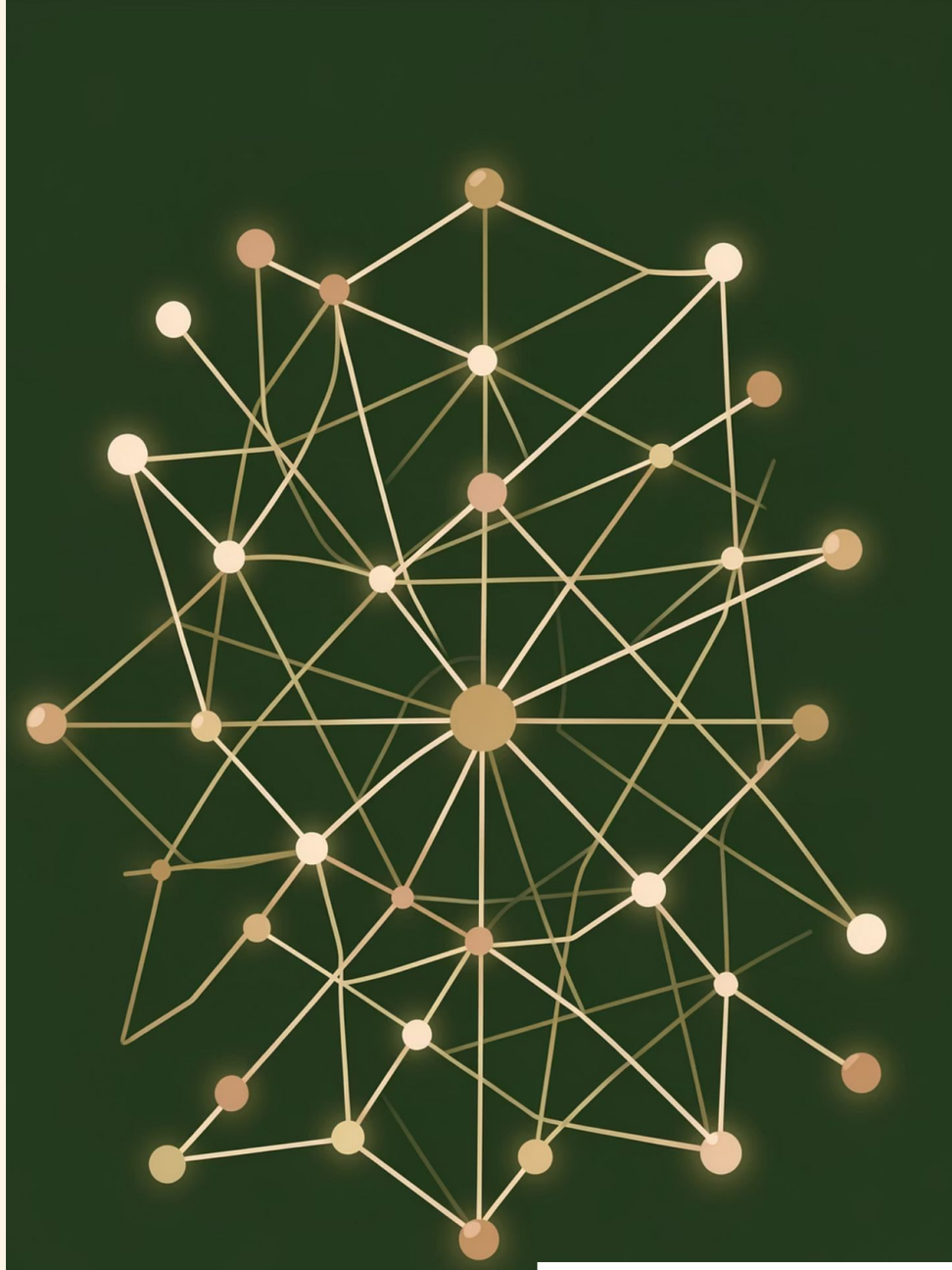
# Nella descrizione della PR:
"Questa PR implementa il
reset password.
closes #22, closes #23"
```

💡 Collegare Issue e PR rende la storia del progetto tracciabile: sai sempre **perché** è stato scritto quel codice.

© 2026 Mattia Nardulli – Tutti i diritti riservati.

Rebase vs Merge: Scegliere la Strategia Giusta

Due approcci, una sola domanda: come integrare il lavoro del tuo team in modo pulito ed efficace? In questa presentazione esploriamo le differenze fondamentali tra **git merge** e **git rebase**, quando usare ciascuno, e le regole d'oro per non rovinare la storia del tuo repository.



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Il Ciclo di Vita di un Branch

Prima di scegliere tra merge e rebase, è fondamentale capire perché questi comandi esistono. Quando più sviluppatori lavorano sullo stesso progetto, i branch divergono inevitabilmente. Git offre due strategie distinte per ricongiungerli, ognuna con implicazioni diverse sulla **storia del repository** e sulla **collaborazione**.



Branch Divergono

Ogni sviluppatore lavora sul proprio branch, aggiungendo commit indipendenti.



Necessità di Integrare

Arriva il momento di unire il lavoro: feature completata, bugfix pronto, sprint finito.



Scelta della Strategia

Merge o Rebase? La risposta dipende dal contesto, dal team e dalle policy adottate.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Git Merge: Unire Preservando la Storia

git merge integra due branch creando un **merge commit** — un commit speciale con due genitori che registra esattamente quando e come le due linee di sviluppo si sono ricongiunte. La storia rimane intatta e autentica.

Vantaggi

- Preserva la storia cronologica completa
- Sicuro su branch condivisi e pubblici
- Facile da capire e da annullare
- Mostra chiaramente quando avviene l'integrazione

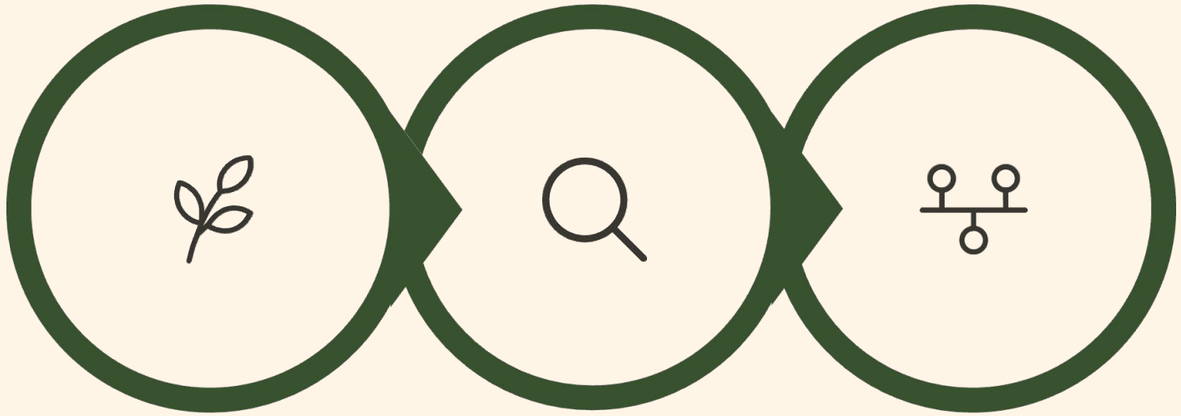
Svantaggi

- Genera merge commit aggiuntivi
- La storia può diventare intricata con molti branch
- Il log git appare meno lineare
- Più difficile leggere la progressione del progetto

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Git Rebase: Riscrivere la Storia

git rebase sposta i commit del branch corrente *sopra* la punta del branch di destinazione, come se il lavoro fosse stato fatto partendo da lì. Il risultato è una storia **perfettamente lineare**, senza merge commit — ma attenzione: la storia viene riscritta.



Branch originario

Rebase su main

Storia lineare

Vantaggi

- Storia lineare e leggibile
- Nessun merge commit superfluo
- git log più pulito e navigabile
- Facilita code review e bisect

Svantaggi

- Riscrive gli hash dei commit
- Pericoloso su branch condivisi
- Conflitti da risolvere più volte
- Richiede maggiore esperienza

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Merge vs Rebase: Confronto Diretto

La scelta tra merge e rebase non è mai assoluta — dipende dal contesto. Questa tabella riassume i criteri principali per orientare la decisione nei casi più comuni durante lo sviluppo quotidiano.

Criterio	🔀 git merge	🔁 git rebase
Storia del repository	Preserva tutto, con merge commit	Riscrive, storia lineare
Branch condivisi (main, develop)	Sempre sicuro	Mai usare
Feature branch privati	Accettabile	Preferibile
Pull Request / Code Review	Comune e consigliato	Ottimo per pulire prima del merge
Risoluzione conflitti	Una sola volta al merge	Potenzialmente per ogni commit
Leggibilità del log	Storia fedele ma complessa	Log pulito e lineare
Quando usarlo	Integrazione finale, hotfix, PR	Aggiornare feature branch, squash

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Comandi Pratici Essenziali

Ecco i comandi fondamentali da conoscere. Ogni riga ha un caso d'uso preciso — padroneggiarli significa avere pieno controllo sul proprio flusso di lavoro Git.

Merge classico

```
git checkout main
git merge feature/login
```

Integra il branch `feature/login` in `main`, creando un merge commit che conserva la storia completa.

Rebase su main

```
git checkout feature/login
git rebase main
```

Riposiziona i commit del tuo branch sopra l'ultimo commit di `main`, ottenendo una storia lineare.

Rebase interattivo

```
git rebase -i HEAD~3
```

Apre un editor per modificare, unire (squash), rinominare o eliminare gli ultimi 3 commit prima di condividerli.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Non fare rebase su branch condivisi.

> Il rebase riscrive la storia dei commit, cambiando i loro hash identificativi. Se altri sviluppatori hanno già scaricato quei commit sul loro repository locale, si troveranno con una storia divergente e incompatibile — causando confusione, perdita di lavoro e conflitti difficili da risolvere.

Rebase OK su...

- Feature branch locali non condivisi
- Branch personali prima di aprire una PR
- Commit non ancora pubblicati su remoto

Rebase MAI su...

`main`, `develop`, `release`

- Branch già condivisi con il team
- Commit già presenti su origin/remoto

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Takeaway: La Strategia Giusta per Ogni Situazione

Non esiste una risposta universale tra merge e rebase — esistono **buone pratiche da applicare con consapevolezza**. Porta questi principi nel tuo workflow quotidiano.

Usa Merge per...
Integrazioni finali, Pull Request, branch condivisi. La storia resterà autentica e tracciabile.

Usa Rebase per...
Aggiornare branch locali, pulire commit prima di una PR, mantenere una storia leggibile.

Ricorda sempre...
Mai rebase su branch condivisi. Comunica con il team le policy adottate nel progetto.

Prossimi passi
Pratica in un repository sandbox, sperimenta entrambe le strategie, leggi la documentazione ufficiale di Git.



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it



Esercizio in Coppia – Git Remoto Completo

Dal Repository alla Collaborazione Reale: un percorso guidato attraverso **push, pull, fetch, branch, merge, conflitti e rebase**. Git come si usa davvero, lavorando insieme su un repository condiviso.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

TC-Web Technology Consulting

Prerequisiti e Setup Iniziale

Prima di iniziare, verificate che entrambi abbiate tutto il necessario. Questo laboratorio funziona solo se la coppia è sincronizzata fin dal primo momento.

🖥️ Git Installato

Git deve essere installato e configurato con `user.name` e `user.email` validi su entrambe le macchine.

🐙 Account GitHub

Entrambi devono avere un account GitHub attivo. Annotatevi il vostro username: vi servirà per l'invito come collaboratore.

Studente A — Owner

Crea il repository, invita il collaboratore, gestisce il merge finale su `main`.

Studente B — Collaboratore

Accetta l'invito, lavora sui branch, simula il conflitto e gestisce il rebase interattivo.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Creazione e Configurazione del Repository

Il primo passo: mettere in piedi il campo di lavoro condiviso.

01

Solo Studente A

Crea un repository pubblico su GitHub chiamato `git-lab-coppia`, inizializzalo con `README.md`, poi vai in **Settings › Collaborators** e invita Studente B tramite il suo username GitHub.

02

Solo Studente B

Accetta l'invito ricevuto via email o notifica GitHub. Da questo momento sei ufficialmente collaboratore con accesso in scrittura al repository.

03

Entrambi

Clonate il repo: `git clone https://github.com/studenteA/git-lab-coppia.git`. Poi verificate con `git remote -v` e `git config --list`.

Verifica: Entrambi vedete la stessa cartella locale con il file `README.md`?



© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Push e Pull Base

Il ciclo fondamentale di Git: modificare, committare, condividere. Imparate a coordinarvi senza calpestare il lavoro dell'altro.

Studente A — Prima

Crea `studente-a.txt` con il testo: *"Ciao, sono lo Studente A. Sto imparando Git."*

Poi esegui:

- `git add studente-a.txt`
- `git commit -m "feat: aggiunto profilo studente A"`
- `git push origin main`

Studente B — Risponde

Esegui `git pull origin main` e verifica che `studente-a.txt` sia comparso. Poi crea `studente-b.txt` con testo analogo, committalo e pushalo.

Infine Studente A esegue di nuovo `git pull` per ricevere il file di B.

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Fetch vs Pull: Capire la Differenza

Molti usano `git pull` senza sapere cosa fa davvero. Questa fase svela il meccanismo interno. **Durata stimata: 10 minuti.**

Studente A modifica il `README.md` aggiungendo la riga *"Progetto creato durante il corso ITS Git"*, poi fa commit e push.

Studente B — invece di fare subito `git pull` — esplora passo dopo passo:

1

git fetch origin

Scarica le informazioni remote **senza modificare** i file locali. Puoi vedere cos'è cambiato senza ancora applicarlo.

2

git log origin/main --oneline

Osserva il nuovo commit di A nel log remoto. I tuoi file locali sono ancora invariati!

3

git diff HEAD origin/main

Visualizza esattamente le differenze tra il tuo stato locale e il remoto, riga per riga.

4

git merge origin/main

Applica le modifiche scaricate. `fetch` + `merge` = `pull`. Ora lo sai!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Branch Remoti e Collaborazione Parallela

I branch sono il cuore del lavoro in team. Permettono di sviluppare funzionalità in isolamento senza rischiare di rompere `main`.

Studente A — Crea il Branch

- `git switch -c feature/lista-spesa`

Crea `lista-spesa.txt` con 3 elementi

Commit: `"feat: aggiunta lista della spesa"`

- `git push -u origin feature/lista-spesa`
- Comunica il nome del branch a B

Studente B — Contribuisce

`git fetch origin` poi `git branch -r`

- `git switch feature/lista-spesa`
- Aggiunge 2 elementi alla lista, commit e push

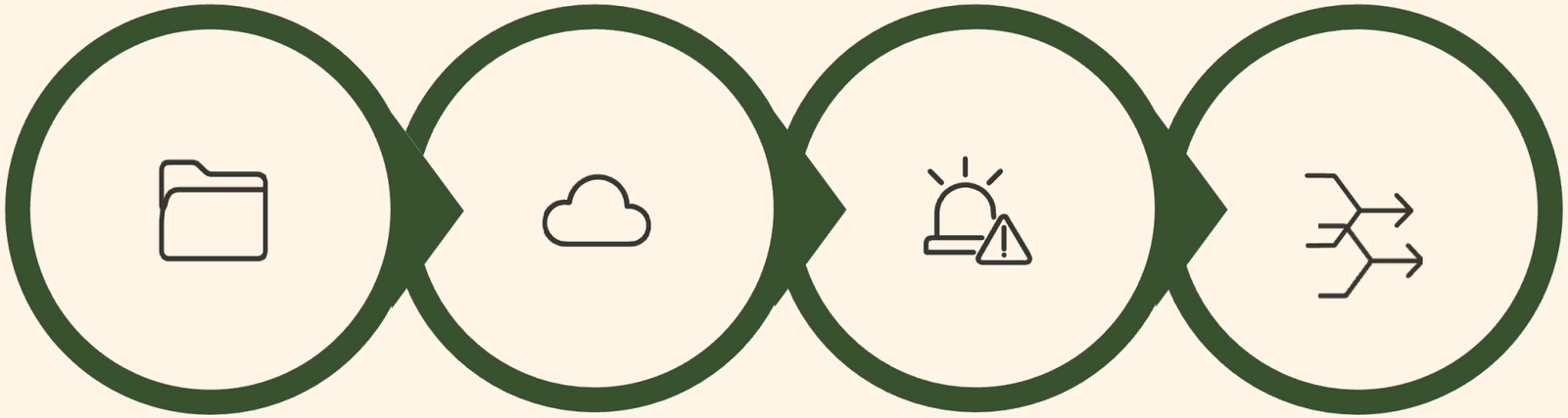
Studente A — Merge su Main

- `git pull origin feature/lista-spesa`
- `git switch main`
- `git merge feature/lista-spesa`
- `git push origin main`

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Merge Conflict Intenzionale

Questa è la fase più realistica del laboratorio: simulate il conflitto più comune nei team reali. La regola: **entrambi modificate la stessa riga dello stesso file.**



Studente B dovrà aprire `descrizione-progetto.txt`, trovare i marcatori `<<<<<<< HEAD` / `=======` / `>>>>>>>`, decidere insieme la versione unificata, rimuovere i marcatori e poi eseguire `git add` → `git commit` → `git push`.

> ⚠️ **Attenzione:** I marcatori `<<<<<<<` non devono **MAI** finire in produzione. Verificate sempre i file prima del push!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Rebase Interattivo: Una History Pulita

Un buon progetto ha una storia leggibile. Il rebase interattivo vi permette di trasformare commit "sporchi" in un unico commit significativo.

Studente B — Squash dei Commit

Crea il branch `feature/note-personali` e fai 4 commit disordinati su `note.txt`: *"wip"*, *"fix"*, *"ancora fix"*, *"ok ora va"*. Poi:

- `git rebase -i HEAD~4`

Lascia `pick` sul primo, cambia gli altri 3 in `squash`

Scrivi il messaggio finale: *"feat: aggiunte note personali dello studente B"*

`git rebase origin/main` per aggiornare il branch

- `git push --force-with-lease origin feature/note-personali`

Studente A — Nel Frattempo

Aggiungi una modifica qualsiasi a `main` e pushala. Questo simula l'avanzamento del progetto mentre B lavora sul suo branch.

Poi, dopo il push di B:

- `git fetch origin`

Esamina il log del branch di B con `git log --oneline`

- Commentate insieme: come appare la history prima e dopo lo squash?

Attenzione: Usa `--force-with-lease` — mai `--force` semplice su branch condivisi. E il rebase va usato **solo** su branch personali!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

Ce l'avete Fatta!

Avete completato un percorso completo di collaborazione Git reale. Ecco tutto quello che avete imparato e praticato insieme.



Push & Pull

Il ciclo base per sincronizzare il lavoro in team senza sovrascrivere le modifiche altrui.



Fetch vs Pull

Capire la differenza vi rende sviluppatori più consapevoli e meno soggetti a sorprese.



Branch Remoti

Lavorare in parallelo su feature separate è la base del flusso professionale con Git.



Merge Conflict

I conflitti fanno paura solo la prima volta. Ora sapete riconoscerli e risolverli con metodo.



Rebase Interattivo

Una history pulita è un regalo per il vostro futuro team. Squash dei commit sporchi in messaggi significativi.

Prossimi passi: Esplorate le **Pull Request** su GitHub, i **workflow Git Flow** e la **CI/CD**. Il laboratorio di oggi è la base di tutto!

© 2026 Mattia Nardulli – Tutti i diritti riservati.
TC-Web srl - Via Schiaparelli 14 - Torino https://www.tc-web.it

GRAZIE A TUTTI…..

Via Schiaparelli 14 - 10148 Torino

https://www.tc-web.it

Mail: franco.grivet@tc-web.it

© 2026 Mattia Nardulli – Tutti i diritti riservati.